

Cassiopeia Documentation

What is Cassiopeia?

Cassiopeia (which we fondly call Cass) is a framework for pulling and working with data from the [Riot API](#). Cass differentiates itself from other API wrappers by taking a page from one of Cassiopeia's quotes, "I'll take care of everything." Our main goal is to make your life (and ours) as developers *easy*.

Cass is composed of three key pieces:

1. An *interface* for pulling data from the Riot API.
2. A *type system* of classes for holding and working with the data pulled from Riot.
3. *Caches and databases* to temporarily and permanently store that data.

Together, these three pieces provide the user experience we desire. Scroll down for a quick example of how Cass works, what Cass does for you as a user, and information about contributing.

Why use Cass?

- An excellent user interface that makes working with data from the Riot API easy and fun.
- "Perfect" rate limiting.
- Guaranteed optimal usage of your API key.
- Built in caching and (coming) the ability to easily hook into a database for offline storage of data.
- Extendability to non-Riot data. Because Cass is a framework and not just an API wrapper, you can integrate your own data sources into your project. Cass already supports Data Dragon and the `champion.gg` API in addition to the Riot API.
- Dynamic settings so you can configure Cass for your specific use case.

An Example

We will quickly and efficiently look up the champion masteries for the summoner "Kalturi" (one of the developers) and print the champions he is best at. If you just want a quick look at how the interface looks, feel free to just read these three lines and skip the explanation. The explanation explains how the three bullet points above fit together and allow this code to be run.

```
kalturi = Account(name="Kalturi", tagline="NA1", region="NA").summoner
good_with = kalturi.champion_masteries.filter(lambda cm: cm.level >= 6)
print([cm.champion.name for cm in good_with])

# At the time of writing this, this prints:
['Vel'Koz', 'Blitzcrank', 'Braum', 'Lulu', 'Sejuani']
```

The above three lines are relatively concise code, and if you know what lambdas and list comprehensions are then it will likely be readable. However, there is a deceptive amount of logic in these three lines, so let's break it down. (If you don't understand everything immediately, don't worry, that's why you're using Cass. You don't have to understand how everything works behind the scenes, you just get to write good code.)

 [latest](#) ▼

```
Account(name="Kalturi", tagline="NA1", region="NA")
```

First, we create an Account with a name and tagline. This code **does not** make a call to the Riot API, it merely creates an Account object where the name and tagline fields are populated.

```
kalturi = Account(name="Kalturi", tagline="NA1", region="NA").summoner
```

The Account's `.summoner` field is accessed and an API call is made to the Riot API to get the rest of the **account** information based on the name and tagline (i.e. the puuid is pulled from the Riot API). Then a Summoner object is stored in the `kalturi` variable. Note that the code has not yet made an API call for the **summoner**.

```
... = kalturi.champion_masteries ...
```

Next we ask for the champion masteries for `kalturi` by running `kalturi.champion_masteries`. This creates an un-instantiated list which will contain champion masteries if any item in it is accessed.

```
good_with = perks.champion_masteries.filter(lambda cm: cm.level >= 6)
```

Third, the `.filter` method is called on the list of champion masteries. `filter` is a python built-in that operates on a list and filters the items in it based on some criteria. That criteria is defined by the `lambda` function we pass in.

A `lambda` is a quick way of defining functions in-line without using the `def` statement. In this case, `lambda cm: cm.level > 6` takes in an object and assigns it to the variable `cm`, then it returns `cm.level > 6`. So this `lambda` will return `True` for any champion mastery whose mastery level is greater than or equal to 6.

The `.filter(lambda cm: cm.level > 6)` therefore operates on the list of champion masteries. When the list is iterated over, the champion masteries are queried. This requires a summoner id, which is pulled from `kalturi.id`, and the Riot API is queried for Kalturi's champion masteries. With the champion mastery data pulled, `.filter` then filters the list looking for all champion masteries with mastery level 6 or higher.

```
print([cm.champion.name for cm in good_with])
```

Finally, the third line prints a list of the champion names for those champions.

Together these three lines illustrate the concise user interface that Cass provides, the way in which the data can be used, when the data is pulled (queried).

Contributing

Contributions are welcome and we have an entire [page](#) devoted to ways in which you can help us with Cass.

Overview

- [Using Cassiopeia](#)
 - [Methods and Class Constructors](#)
- [Setup](#)
 - [Install using pip](#)

- [PyCurl Issues](#)
 - [Alternative library support](#)
 - [Install from Source](#)
 - [Setting Your API Key and Other Settings](#)
- [Settings](#)
 - [Globals](#)
 - [Data Pipeline](#)
 - [Riot API](#)
 - [Logging](#)
 - [Plugins](#)
- [How Cass Works](#)
 - [Two Interfaces](#)
 - [Settings](#)
 - [Ghost Loading](#)
 - [Data Pipeline](#)
 - [Searchable Containers](#)
 - [Match Histories Work Slightly Differently](#)
- [Data Pipeline](#)
 - [Defining Components in your Settings](#)
 - [Components](#)
- [Plugins](#)
 - [Simple KV Disk Store](#)
- [Contributing](#)
 - [Things we need help with!](#)
- [Regions](#)

Top Level APIs

- [Settings](#)
- [Data and Enums](#)
- [Champions](#)
- [Champion Masteries](#)
- [Items](#)
- [Language Strings](#)
- [Leagues](#)
- [Locales](#)
- [Maps](#)
- [Matches](#)
- [Patch](#)
- [Profile Icons](#)
- [Realms](#)
- [Runes](#)
- [Status](#)
- [Spectator](#)
- [Summoners](#)
- [Accounts](#)
- [Summoner Spells](#)

- [Versions](#)

Index and Search

- [Index](#)